

Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial

Instituto Tecnológico de Buenos Aires



Grupo G92

Integrantes:

- Barnatán, Martín Alejandro (64463)
- Garrós, Celestino (64375)
- Pedemonte Berthoud, Ignacio (64908)
- Weitz, Leo (64365)

Fecha de Entrega: 22/06/2025

2025

Tabla de Contenidos

1. Introducción.....	3
2. Modelo Computacional.....	3
2.1 Dominio.....	3
2.2 Lenguaje.....	4
3. Implementación.....	4
3.1 Frontend.....	4
3.2 Backend.....	5
3.3 Dificultades Encontradas.....	6
4. Futuras Extensiones.....	6
5. Conclusiones.....	6
6. Referencias.....	7

1. Introducción

El presente trabajo consistió en el desarrollo de un compilador con el objetivo de permitir consultas SQL en formato JSON en el marco de la materia Autómatas, Teoría de Lenguajes y Compiladores. Se buscó principalmente brindar una opción más sencilla al usuario para realizar consultas a una base de datos PostgreSQL.

El compilador fue desarrollado en el lenguaje de programación C. Una de las principales herramientas empleadas fue Flex para la definición y generación del analizador léxico, el cual se encarga de identificar los distintos componentes léxicos del lenguaje de entrada. Otra importante herramienta utilizada fue Bison, que se utilizó para construir el analizador sintáctico para validar que la estructura del JSON siga la gramática definida, además de construir las estructuras internas necesarias para la posterior generación del código SQL.

El proceso se separó en tres etapas. La primera consistió en la propuesta y diseño de la idea principal, donde se pensaron requerimientos, propuesta, ejemplos y casos de prueba. En la segunda etapa, se llevó adelante el desarrollo del frontend, que representa “el frente” del compilador, es decir el analizador léxico y sintáctico. El principal objetivo de esta etapa fue la construcción del AST (Árbol de Sintaxis Abstracta) de la entrada. Finalmente, la tercera etapa consistió del terminado de algunos aspectos del frontend y validaciones, para luego generar el código deseado por el usuario.

2. Modelo Computacional

El proceso de desarrollo del compilador se basó en tres etapas, como se mencionó anteriormente. En el proceso de diseño, dos aspectos fundamentales que se determinaron fueron el dominio y el lenguaje.

2.1 Dominio

El dominio del compilador desarrollado en este proyecto se centró en la construcción queries válidas SQL que pudieran ser ejecutadas por el usuario en bases de datos PostgreSQL a partir de estructuras JSON. Se diseñó un lenguaje de entrada centrado en la búsqueda de datos, con cláusulas como **SELECT**, **WHERE**, **ORDER BY**, **GROUP BY**, y también funciones de agregación. También se hizo foco en la posibilidad de realizar consultas anidadas.

Como se mencionó previamente, el enfoque estuvo únicamente en consultas y no en otras operaciones como inserciones (**INSERT**), modificaciones (**UPDATE**) o eliminaciones (**DELETE**). Esta decisión permitió reducir la complejidad del proyecto para centrarse en el correcto modelado de las estructuras de las consultas, y su traducción a SQL.

Por otro lado, se le dio principal atención a la posibilidad de definir consultas como estructuras anidadas, ya que se consideró como uno de los principales aspectos que agrega valor al usuario. Al poder realizarlas de esta forma en una única estructura JSON, se provee una mayor claridad y simplicidad tanto en la redacción de las consultas como en su lectura y revisión.

2.2 Lenguaje

El lenguaje de entrada está basado en formato JSON, como se mencionó previamente, lo que permite escribir consultas SQL con una estructura jerárquica y mucho más cómoda para el usuario. A cada elemento del lenguaje SQL se lo mapeó con una clave específica dentro del JSON, como se muestra a continuación en el caso de **FROM** y de los atributos:

```
SELECT attr_1, attr_2, ..., attr_n FROM table_1;

{
  "from": ["table_1"],
  "attributes": [{"name": "attr_1"}, {"name": "attr_2"}, ...,
  {"name": "attr_n"}]
}
```

El primer campo presentado es obligatorio, ya que salvo casos límites ninguna consulta puede ser ejecutada sin él (si no se especifica el campo "attributes" se asume que se quiso seleccionar todos). Se puede apreciar en el ejemplo una mayor legibilidad de los elementos de la consulta, por lo que se le simplifica al usuario el desarrollo de casos de mayor complejidad. Otro aspecto importante es que no se requiere de un orden específico para los campos, permitiendo una mayor libertad para la composición de las queries.

A nivel sintáctico, se implementaron las siguientes representaciones, teniendo en cuenta la simplificación en la complejidad expresada anteriormente:

- Selecciones simples de columnas.
- Filtros por condiciones booleanas combinables (**AND**, **OR**, **NOT**).
- Ordenamientos (**ORDER BY**) por múltiples criterios.
- Funciones de agregación, como por ejemplo **COUNT**.
- Alias de columnas y expresiones.
- Consultas anidadas y subconsultas como operandos de cláusulas.
- Cláusula **HAVING** para filtrar datos por sobre agrupaciones previas.

El lenguaje de salida entonces se corresponde a un subconjunto del lenguaje SQL, donde se basó principalmente el diseño para la aplicación sobre una base de datos Postgres. La generación de este código se realiza una vez que se validó la estructura del JSON de entrada y que se construyó el árbol de sintaxis.

3. Implementación

Para la implementación del compilador se utilizó como base el proyecto [Flex-Bison-Compiler](#) (sobre la rama development), que facilitó el uso de las herramientas Flex (para el análisis léxico) y Bison (para el análisis sintáctico), como ya se describió previamente.

3.1 Frontend

Para desarrollar el frontend, el énfasis estuvo principalmente en la definición de una gramática formal que representara el lenguaje deseado. De esta manera se definieron como símbolos terminales: strings, enteros, floats, booleanos y diversos tipos de tokens (enteros

que forman parte de un enum). Por otro lado, se definieron como no terminales las estructuras de más alto nivel:

- Las cláusulas **WHERE**, **FROM**, etc.
- Los distintos tipos de condiciones y los valores que algunas cláusulas pueden tomar.
- El símbolo inicial, definido como un **program**, del cual se deriva la estructura JSON que acepta el lenguaje.

Además, se intentaron modularizar y generalizar las acciones sintácticas de Bison lo más posible para facilitar su lectura en la gramática. Con este objetivo en mente, se crearon estructuras generales tales como **ClauseArgsList**, que representa una lista de argumentos válidos de una cláusula particular, o **ClauseValue** que contiene un valor válido para una cláusula particular. Esto se pudo realizar en todos los casos excepto con la cláusula **WHERE**, la cual por su complejidad se abordó de una manera distinta al resto.

Desde el frontend fue posible validar muchas estructuras y descartar otras tantas consideradas inválidas dentro del lenguaje. Sin embargo hubo un par de casos que, debido a la estructura de la propia gramática y sus producciones, se debieron postergar para validaciones semánticas del backend.

3.2 Backend

La primera etapa del desarrollo backend del compilador consistió del análisis del lenguaje elaborado previamente para poder determinar adecuadamente los elementos de la tabla de símbolos, donde se guardó información semántica relevante de los identificadores para realizar validaciones.

En primer lugar, como en el alcance del proyecto no se abordó la definición de datos (como ser tablas) no es posible validar por ejemplo columnas existentes o inexistentes, ni los tipos de dos columnas distintas. Es por esto que no hubo necesidad de agregar ningún elemento de los campos **FROM** o **ATTRIBUTES** en dicha tabla. En cambio, para el caso de las queries auxiliares se validó que efectivamente existieran y no se pudiera realizar un llamado a una indefinida, además de verificar que no se pudieran redefinir. Es por esto que se agregaron dos listas a la tabla de símbolos, una para las queries auxiliares definidas y otra para las usadas.

Previo a la generación de código, se realizaron además otras validaciones utilizando tanto el AST como la tabla para verificar:

- La cláusula **FROM** es obligatoria en toda consulta (que si bien en PostgreSQL no es obligatorio, es un edge case que no se consideró relevante).
- Se permite el uso de **JOIN** simples para combinar tablas, respetando las restricciones estructurales del lenguaje.
- No se permite la repetición de cláusulas dentro de una misma consulta (por ejemplo, múltiples **SELECT**, **FROM**, etc.).
- Se permite el uso de la cláusula **GROUP BY**, incluyendo múltiples columnas, en conjunto con funciones de agregación.
- Las condiciones dentro de la cláusula **HAVING** solo se permiten si existe una cláusula **GROUP BY** asociada.

- No se permite declarar una consulta auxiliar (query nombrada) dentro de otra consulta auxiliar.
- No se permiten comparaciones directas con el valor `NULL`, ni el uso de operadores relacionales (`<` o `>`) en dichas comparaciones.

Para la generación de código, se reutilizó la función output de la calculadora en el proyecto provisto por la cátedra, removiendo la indentación que no era necesaria en este caso. Se definió enviar el resultado a la salida estándar, ya que se consideró que sería el resultado óptimo en cuanto a comodidad para un usuario que buscara convertir su estructura JSON en una query SQL.

3.3 Dificultades Encontradas

Una de las principales dificultades encontradas en el desarrollo del compilador fue la cláusula `WHERE` en el frontend. Dada la amplitud que SQL ofrece para la construcción de condiciones, el modelado de esta flexibilidad fue un problema en dicha etapa. En la búsqueda de que el lenguaje aceptado sea útil para el usuario, se debió tratar esta cláusula como un caso aparte en la implementación de la gramática, y se buscó darle la mayor cantidad de funcionalidades posible, dentro del alcance del proyecto.

Otra dificultad surgió durante la etapa de desarrollo del backend, donde se encontró la necesidad de implementar un scope. Esto fue necesario para la validación de que existiera la correspondiente cláusula `GROUP BY` en el caso de la utilización de funciones de agregación dentro de la cláusula `WHERE`. Al manejar queries auxiliares dentro de una consulta principal, este enfoque fue necesario para que la validación se hiciera dentro de cada una, sin interferir con el resto.

4. Futuras Extensiones

Siendo que el lenguaje intenta aceptar un subdominio del mundo SQL, existen casi infinitas direcciones en las que podría expandirse para aceptar cada vez más consultas distintas, teniendo en cuenta la dificultad mencionada en la sección anterior. Una opción que resultó particularmente interesante fue la posibilidad de expandir el lenguaje e incluir un Data Definition Language que permitiera a un usuario no simplemente hacer consultas SQL, sino también crear sus propias tablas. Con esta nueva funcionalidad sería posible validar todavía más precisamente las consultas que un usuario haga sobre sus propias tablas, explotando al máximo el potencial del lenguaje desarrollado.

5. Conclusiones

Como conclusión del proyecto, se pudo llevar a cabo el desarrollo de un compilador, pasando por las tres etapas propuestas por la cátedra, comenzando con la idea y luego realizando tanto la parte de frontend como backend. Para el caso del lenguaje tomado, se atravesó una segunda etapa con mayor cantidad de complicaciones debido a la amplitud de funcionalidades de SQL para la cláusula `WHERE`. El último tramo del desarrollo se simplificó en gran medida dado a que no se debió manejar runtime, y muchas validaciones ya se habían realizado en la sección previa.

A su vez, el proyecto permitió una visualización más concreta de las temáticas estudiadas en la materia, permitiendo a los integrantes elegir el dominio y los lenguajes, y desarrollar los análisis sintácticos, léxicos y semánticos, así también como realizar la generación de código. El proyecto base provisto por la cátedra fue una herramienta fundamental para el cumplimiento de la tarea asignada, aunque representó un nuevo desafío ya que se basaba en tecnologías con las que nunca se había trabajado previamente (Flex y Bison).

6. Bibliografía

1.  (2025-03-13, v3.0.6) Proyecto Especial
2.  (2025-03-13, v0.1.0) Modelo Computacional
3.  (2024-09-25, v0.2.0) Análisis Léxico
4.  (2024-05-08, v0.1.0) Análisis Sintáctico
5.  (2024-05-08, v0.1.0) Análisis Sintáctico
6.  (2024-09-25, v0.2.0) Análisis Semántico
7.  (2024-05-16, v0.1.0) Generación de Código